Your Data, Your Rules, One Query: Extending the Decentralized Web

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

ActivityPub (Mastodon) and ATProto (Bluesky) are running, non-cryptographic social protocols—no blockchain, no gas fees, no wallet-gated feeds. But each defines a rich protocol and then steps outside it: the server knows what a post, a follow, or a like is, and special-cases them for performance. The protocol is the spec; the implementation cheats around it.

We present **web10**, a user-owned-data platform whose node contains *no social primitives at all*—no feed, follow, post, group, or message endpoint. The entire social application (feeds, groups, direct messages, discovery) is composed, 100%, from generic documents, groups, roles, and a query engine: every database call goes through the same generic path, with no platform-specific shortcut. Access is governed by fine-grained, revocable, group/role-scoped contracts enforced at the query layer, so the generic path stays secure at an index-lookup cost rather than a scan. And because that generic path is fast enough, there is no reason to cheat: a feed that a naive implementation loads with **267 network requests** loads in **one** web10 query-engine call. A full social platform, built nothing but generic queries, that runs at commercial grade—and a node general enough to host a messaging app, a music app, or a shop without changing the platform.

1 INTRODUCTION

The dominant social platforms solve engagement extremely well and ownership extremely poorly. A user's posts, messages, and relationships live in the platform's database under the platform's access model. The user can adjust a handful of visibility settings the platform chose to expose; they cannot redefine who a "friend" or a "member" is, cannot contract access per-service, and cannot take their data—let alone their audience— elsewhere. When the platform changes its rules or goes away, the user's data goes with it.

For a subset of users this is not an abstraction—it is a paycheck. Creators and influencers build their income on their social graph: their audience is the asset, and the platform that hosts it can de-platform them, shadowban their reach, reprice their distribution, or take a larger cut, at will, with no recourse. Trust in the platform is therefore a financial dependency, not a preference. web10 removes that dependency by construction: delivery is guaranteed by architecture (the feed is a query over the creator's own data, not a platform-controlled ranking), and the audience is portable because it lives under the creator's key, not the platform's.

Cloud infrastructure solved the authorization half of this problem long ago. AWS IAM and its descendants give operators fine-grained, role-based, service-scoped, revocable access control with cryptographic token verification. But that model lives at the *infrastructure* boundary: the cloud provider owns the data plane, and the end user is invisible to it. Nobody has brought the IAM model to the *user* boundary of a social platform—where the user, not the platform, is the principal that holds the contracts.

The decentralized social web has real, running systems, and web10 builds on that lineage rather than against it. **ActivityPub** (Mastodon) and **ATProto** (Bluesky) are live, non-cryptographic protocols with large user bases—deliberately no blockchain, no gas fees, no wallet-gated feeds, which is the practical choice web10 shares. Their gap is the access model: authorization stops at follow/block and collection-level control, with no fine-grained group/role contracts and no request/approve/assign lifecycle, and their data models are shaped to one app (a microblog or a post graph) rather than a platform that hosts many. **Solid** goes further on ownership (personal data stores, WebACL) but its container-level ACLs still lack a group/role primitive, and its per-Pod storage makes a cross-user feed query intractable, so its social applications never reached the scale of Mastodon or Bluesky. **Cloud IAM** has exactly the permission language a social platform needs—principals, roles, service-scoped operations, revocable grants—but it lives at the infrastructure boundary, where the provider owns the data and the end user is invisible.

What none of them do is *live* in their protocol. Each defines rich primitives and then bypasses them—the server special-cases posts, follows, and likes for performance, so the protocol is a layer conformed to at the edges, not the path every request takes. web10 does the opposite. Its node has no social primitives, and its application never steps outside the generic path: every database call—feed, follow, message, group, discovery—goes through the same documents, groups, roles, and query engine, with no platform-specific shortcut. And because that generic path is fast enough (a 267-request feed loads in one), there is no reason to cheat. That purity—and the access model and columnar backend that make it safe and fast—is this paper.

web10 is a user-owned-data platform that makes that move. Every user's data lives under their own key in a single columnar store; every access decision is a check against two user-held contract layers; and the social application (feeds, groups, direct messages, discovery) is composed 100% from generic primitives, with no app-specific surface in the node. This paper makes three contributions:

(1) **A fully generic node, used fully.** The node exposes only universal primitives—documents, groups, roles, services, and a query engine—and contains no social endpoints. The entire social application is composed from those primitives, every database call through the generic path, with no special-case fast path. The same node hosts a messaging app, a music app, or a shop without platform changes (§2, §4).
(2) **An access model that keeps the generic path safe** (§2): an outer *app trust* layer (per-origin, per-service operation grants)

and an inner *person trust* layer (group roles as per-service permission maps over three nested principal classes). Enforced at the query layer, the role check never touches the large document table—fine- grained control at an index-lookup cost, not a scan (§3).

(3) **Proof that pure-generic is fast enough** (§4): a running social platform whose feed—naively 267 network requests—loads in a single query-engine call, every row access-checked against the reader's role grants. The generic path needs no shortcut because it is already fast.

The remainder of this paper: §2 defines the generic primitives and the access model; §3 shows how access is enforced over one columnar table at zero scan penalty; §4 presents the running social platform—composed entirely of generic queries—and its 267-to-1 feed as the demonstration; §5 positions web10 against the decentralized social protocols and cloud IAM; §6 concludes.

2 THE ACCESS MODEL

2.1 Two trust layers

Every request in web10 names an *app* (the calling origin), a *principal* (the acting user), a *group* G, a *service* S (a data namespace such as `posts` or `comments`), and an *operation* O (e.g. `readAll`, `create`). Authorization asks two independent questions, and *both* must pass:

Layer	Question
App trust (outer wall)	Can this *app* perform O on S?
Person trust (inner wall)	Can this *principal*'s effective role in G grant O on S?

The app contract is *infrastructure trust*: which origins may touch which services, enforced by CORS in the browser and re-checked server-side on every call. The group role is *social trust*: which people may do what inside a group. The split matters because the two layers change at different rates and for different reasons—a user re-points an app's access without leaving any group, and leaves a group without touching any app's access.

2.2 Roles are per-service permission maps

A role is a named bundle of per-service permissions—a map from service to the operations it grants:

```
{ "name": "editor",
  "permissions": {
    "posts":    ["readAll", "create", "updateOwn"],
    "comments": ["readAll"]
  } }
```

The map *is* the scope. `'*'` wildcards all document services; a reserved key `'group'` carries *structural* operations on the group itself (`manageRoles`, `assignRoles`, `deleteGroup`, join/member operations), while *content* operations—including moderation such as `hideAll`—live under the service key, because they act on documents, not on the group's shape.

Two properties keep the model small. First, the role map is the *same shape* as the app-contract permission object: there is one permission language across both trust layers. Second, a person holds *exactly one role* per group; the per-service map makes a single role fully expressive over any (principal, service, operation) matrix, so there is no multi-role stacking to reason about. Roles are generic—a group defines whatever roles fit its purpose (`owner`, `moderator`, `page-curator`, `member`, or `dj`, `listener`)— and the platform never names them.

2.3 Principal classes and effective permissions

Access is granted to *principals*, of which there are three nested classes, broadest to narrowest:

Class	Who
`anyone`	every request, signed in or not
`authenticated`	any valid token (any user, member or not)
`member`	holds a member row in this group

The classes are *reserved keys in the member table*—no separate visibility table exists. A group grants a class a role by storing a member row for the reserved key:

```
group_members:
  (G, 'anyone',        'reader')    -- the group is
       public
  (G, 'authenticated', 'reader')    -- signed-in users
       can read
  (G, 'bob',           'moderator') -- bob moderates
  (G, 'alice',         'owner')     -- alice owns
```

A principal's *effective* permissions in a group are the union of the permission maps of every class they belong to:

$$\begin{aligned}\text{effective}(P, G) \;=\; &\text{grant}(G,\ \texttt{anyone}) \\ \cup\ &\big(P \in \texttt{authenticated}\ ?\ \text{grant}(G,\ \texttt{authenticated}) : \emptyset\big) \\ \cup\ &\big(P \in \texttt{member}(G)\ ?\ \text{grant}(G,\ \text{role}(P)) : \emptyset\big)\end{aligned}$$

The nesting buys a monotonicity invariant for free: a member always sees at least what a signed-in stranger sees, who always sees at least what a signed-out visitor sees. The model cannot express the inversion.

2.4 Publicness is a grant, not a flag

There is no `visibility` column. "Make my group public" is the act of granting `readAll` to `anyone`; "signed-in only" grants it to `authenticated`; "private" grants it to member roles only. Join policy is orthogonal: it controls how a human *becomes* a member (open, request, invite-only) and says nothing about what a non-member bystander can see. A group can be invite-only to join and still publicly readable—the discovery board is exactly that.

This is the sense in which web10 brings IAM to the front: the operations a cloud operator would express as policies—*principal class, service, operation, grant*—are the operations an *end user* expresses about their own data, with the same revocability (a grant is a row; removing it is a tombstone that takes effect on the next read).

3 ENFORCEMENT AT THE QUERY LAYER

3.1 One columnar table

A social feed is an analytical workload: every read is a join across users, groups, and time—filtered, ranked, and paginated—which is OLAP, not OLTP. web10 therefore uses a single columnar store for *all* user data, with no Postgres, no change-data-capture pipeline, and no separate operational database. The "OLTP" operations are translated to the columnar model: an update is a new row with a higher timestamp, a delete is a tombstone row, a point lookup is

a primary-key scan, and a counter is a `count()` at read time. This removes the two-database drift (schema divergence, sync lag) that a Postgres-to-ClickHouse pipeline would introduce, at the cost of embracing the columnar write idioms described below. All structured user content lives in a single ClickHouse table, keyed by `(author_key, doc_id)`; large binary objects (media) live in object storage and are *referenced* from this table (§3):

```
CREATE TABLE documents (
    doc_id          String,
    author_key      String,
    collection_name String,   -- the service: 'posts', '
        comments', ...
    body            String,   -- JSON
    tags            Array(String),
    created_at      DateTime64(3),
    updated_at      DateTime64(3),
    deleted         UInt8 DEFAULT 0
) ENGINE = ReplacingMergeTree(updated_at)
ORDER BY (author_key, doc_id);
```

Updates are inserts with a higher `updated_at` (the engine keeps the latest version); deletes are tombstone rows filtered at read time. Documents attach to groups through a small mapping table, and groups carry their contracts and members in two more small tables. Content is one wide columnar table; *access is a handful of narrow metadata tables.*

3.2 The query boundary: a wall, not a membrane

The genericity claim in §2 has an obvious objection: if any app can write an arbitrary `SELECT` over a multi-tenant store, how does that not leak other users' data? The answer is a constrained query-compilation layer that makes the raw tables *unreachable as inputs*, not merely filtered from the output. We call this a *wall, not a membrane*: the guarantee is not "we filter bad rows out of the result" but "the caller's query can only read pre-filtered views; the underlying tables are simply not addressable."

Why output-side filtering is unsound. The natural design wraps the caller's query and filters the *result*, e.g. `SELECT * FROM (q) WHERE <filter>`. This fails because the caller's query has *already read the raw table* inside the parentheses, and an outer filter cannot undo an aggregation that baked everyone's data in:

```
-- counts every user's comments; the outer group filter
     is too late
SELECT author_key, count(*)
FROM documents WHERE collection_name = 'comments'
GROUP BY author_key
```

The aggregate has already collapsed all authors into counts before any row-level filter could run. Filtering the output of an aggregation is not access control.

Redirecting the input instead. web10 parses the caller's SQL to an AST (never executing the string), then rewrites every table reference to a *boundary CTE*—the service's documents, deduped, joined to `doc_groups`, and filtered to the caller's readable groups for that service, with the block/sharing/hidden exclusions applied. The caller's `SELECT ... FROM posts` now reads that CTE; `posts` is no longer the raw table, it is the caller's own groups and nothing else. Because the raw table never appears in the caller's query, there is nothing to aggregate over, join, or subquery outside the boundary—aggregation, self-joins, and subqueries all stay inside it by construction.

The walk and the backstop. The compilation is five steps: (1) parse to an AST (unparseable or multi-statement input is rejected); (2) type-check (single `SELECT` only—`INSERT`/`DROP`/… rejected); (3) walk every `Table` node, including inside caller CTEs and subqueries, rejecting raw node tables, ungranted services, and table functions (`file()`, `s3()`, `numbers()`—the one non-`Table` escape off the node, which surfaces as an empty-named `Table`); (4) inject the needed boundary CTEs ahead of the caller's CTEs and re-emit from the validated AST; and (5) *round-trip*—re-parse the compiled SQL and require it to come back as exactly one statement, a backstop against a parser mis-read that hides a table reference. String-injection attacks (stacked statements, comment-hidden references, gibberish) are neutralized because the string is resolved into a tree before any check runs; there is no string boundary to jump out of.

Why the database's own RLS does not fit. A reviewer's next question is why not use ClickHouse row-level security or per-account roles. The access boundary here is a *dynamic, membership-dependent JOIN predicate*—"the documents in the groups I am *currently* a member of, where my effective role grants this service." Row-level security in an OLAP engine expresses *static* row predicates bound to an account; it cannot express a predicate that depends on a live membership join evaluated per reader. The boundary therefore cannot live in the storage engine; it must be injected by the compilation layer, per query, per reader. This is the structural reason the security logic sits above the database rather than inside it.

An honest caveat. The guarantee rests on the SQL parser reading the ClickHouse dialect faithfully. An unparseable query is rejected (safe); a mis-parsed one is caught by the round-trip re-parse when it is malformed, though a subtle semantic mis-parse that hides a table reference is the residual risk. The pinned test suite—including an end-to-end run of the compiled SQL against a real store with real membership data, asserting that raw tables 403 in every shape (top-level, caller CTE, subquery) and that aggregation cannot leak past the boundary—is the tripwire. The database account is deliberately *not* a backstop: the boundary CTEs legitimately read the raw tables, so the service account must be able to; the rewriter is the only wall.

3.3 The role check never touches the document table

The read path is the heart of the design:

(1) **Resolve.** Look up the reader's rows in `group_members` (including the reserved principal-class rows) and the role maps in `group_contracts`. These are small, hot, and indexed by their primary keys.
(2) **Compute.** For the requested service, compute the set of group identifiers whose effective role grants `readAll` to the reader.
(3) **Scan.** Run the columnar `documents` scan exactly as it always would, with the cleared-group list as an ordinary filter, unioned with the reader's own `author_key`.

The authorization decision is therefore an index lookup into small metadata tables, not a per-document check inside the scan.

The scan stays a scan. This is the "zero scan-time penalty" property: fine-grained, per-group, per-service control costs the same as the coarsest possible filter, because it *is* a filter list computed once per query.

3.4 The same invariant on every path

The enforcement rule is uniform: *no query returns documents for an* `author_key` *the token does not own, unless group membership grants access* (invariant I3). Every read path—group feed, read-by-id, directory search—applies the identical principal-class gate, and the write side is gated the same way, so a bystander cannot attach content to a group they cannot write. A conformance suite of permission-matrix cases (including replayed-exploit denials) enforces the invariants mechanically; the full suite results appear in the extended version.

3.5 Two data dimensions, one access model

web10 stores user data in two places: structured rows in the columnar store, and large binary objects (images, audio, video) in object storage (MinIO/S3 API). Both are governed by the *same* group/role/I3 model, because the permission gate for an object is a *document*, not the file.

A media object is referenced by a row in `documents` (a `media_refs` array or a `minio` leaf carrying the object key). Access to the bytes is therefore derived from access to that document: the node never hands out a permanent URL. For single-file objects (images, audio, documents) it mints a *presigned URL* with a short TTL—the browser fetches directly from object storage, no middleware hop, and a shared link dies when the URL expires. The gate is the document read that produced the URL; a reader who cannot read the document never gets a URL at all.

Video streaming is the one case where a per-object presigned URL breaks (an HLS video is 100+ small segments fetched continuously over minutes). There the node instead signs a manifest bound to *(reader, document, prefix)* and validates a short-lived JWT on every segment; on each manifest (re)fetch it re-runs the same `can_view_doc` check the read path uses, so a removed or blocked reader loses the stream within one signature TTL. The re-check answers "can this reader see this?" exactly the way the document read did—it is revocation working within a TTL, not a second, stricter model.

The point for the architecture: adding a second storage dimension does not add a second access model. Objects are governed through the documents that reference them, so the group/role contracts and the I3 invariant apply to a creator's video the same way they apply to their post. This is also why the feed's single request can carry resolved media: the query engine's prepare pass mints the per-row object URLs (presigned, or signed manifests for HLS) after the SELECT, reusing the read path's access checks verbatim.

3.6 Data policy, not privacy

A natural objection is that a node which can read its users' content is not really "user-owned"—that true ownership requires the operator to be cryptographically blind. web10 makes the opposite, deliberate choice. The node is a *readable, accountable broker*: content is node-readable by design, because discovery, search, and auditability all require it, and "discoverable" and "hidden from the node" are mutually exclusive. Access is controlled by the permission model (I3) and by *terms*, not by cryptography: the operator who misuses hosted data is legally liable, whereas a system that merely cannot read it offers a weaker promise than one you can take your data from and sue. In short, web10 is a *data-policy* platform, not a *data-privacy* platform—ownership is defined by key-space, portability, and user-held contracts, not by operator-blindness. This is what lets the feed be a fast, global, tunable query in the first place; a privacy-first design could not do cross-user ranking or attribution without consent plumbing.

4 A RUNNING SOCIAL PLATFORM

The strongest evidence for the model is not a benchmark but a product: web10 runs a complete social application—feeds, groups, direct messages, and discovery—entirely on the primitives of §2 and §3, with no app-specific surface in the node (D60). This is the commercial-grade execution of user-owned social data that protocol- first systems such as Solid proposed but did not ship. We highlight the feed, because it is the workload that most stresses the design: a real social feed must be large, tunable, and access-controlled simultaneously.

4.1 The feed as a query

A naive social feed is a stack of N+1 fan-outs: one call per author profile, per post's media, per avatar, per engagement count. Measured on web10's first feed implementation, loading a single page issued **267 network requests**. The fix was not more batching on a bespoke endpoint—it was making the feed an ordinary query over the columnar store, so that one request returns the whole page. Three properties hold at once in that single call:

(1) **It collapses the fan-out.** The 267 requests become **one**: a single query-engine call returns the full page—posts with resolved media and streaming manifests, exact like/comment counts, author profiles and avatars, and attached ads—as one envelope, keyset-cursor paged. The engine's *prepare pass* mints the per-row capabilities (presigned media URLs, ad attachment, author face) after the SELECT, reusing the read path verbatim, so the one request carries everything the old 267 did.

(2) **It is fully user-gated.** Every row in that single response passed the access model on its way out: the query's boundary CTEs apply invariant I3 (no document for an `author_key` the token does not own) plus the reader's effective group/role grants (§2). The result is not an arbitrary batch; it is exactly the set the reader's role clears. Fine-grained social access and a large feed coexist in one query—the conjunction that is this paper's core claim.

(3) **It is tunable—and generic.** The feed's shape—what it contains, how it is ranked and filtered—is a query the *application* writes, not logic hardcoded in the node. A different query is a different feed. Crucially, this goes through the *general* query engine, not a social-specific endpoint: the node never learns what a "feed" is (D60). Any app can issue its own ranked, resolved, access-checked read the same way; the social feed is the reference example.

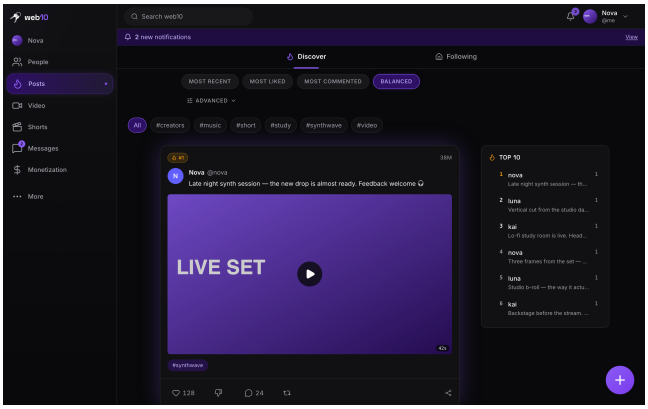


Figure 1: The web10 feed. The reader chooses the ranking (Most Recent / Most Liked / Most Commented / Balanced) and filters (tags); each is a different query over the same columnar store, every result access-checked against the reader's role grants.

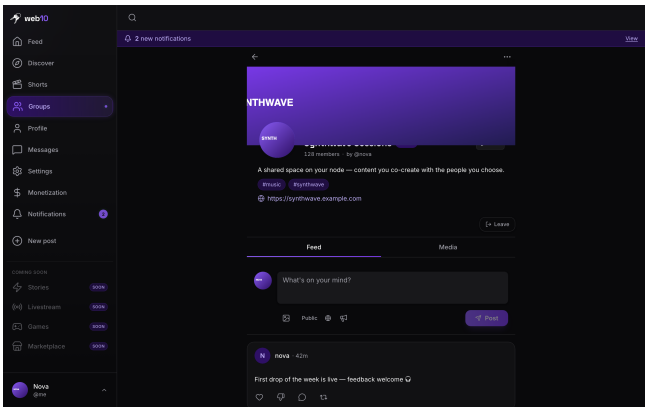


Figure 2: A group—"a shared space on your node." Membership, roles, and per-post visibility (Public / node / group) are the group/role contracts of §2, expressed in the UI.

This is the "because of the columnar backend" point made concrete: Solid's per-Pod architecture makes a cross-user feed query intractable (each Pod is a separate server), so its social applications remain hypothetical. web10's single table makes the feed a first-class, tunable query—267 requests reduced to 1—while the access model stays a filter list with zero scan penalty.

4.2 Durability and extensibility

Because the node exposes only universal primitives and the access model is a per-service role map, the social app is not a one-off demo—it is the hardest instance of a general pattern. The same substrate supports other app shapes (a messaging app, a music app, a shop) by defining new services and roles over the same tables, with no node changes. The social platform demonstrates that the model carries a demanding, real workload; extensibility to other apps follows from the same genericity.

4.3 Scale and latency (extended version)

The deployed system's scale and access-decision latency are reported in the extended version: [**N**] registered users, [**N**] documents across [**N**] services and [**N**] groups, [**N**] access decisions per day; the resolve+compute phase (metadata lookup + effective-role computation) completes in [**X**] ms at p50 / [**Y**] ms at p99, a fraction [**Z**]% of total read latency. The extended version also adds the cost of the per-user key-space model versus a shared table with row-level security, end-to-end flow latencies against a conventional backend, and the full permission-matrix conformance results.

5 RELATED WORK

Decentralized social data. web10 is in the lineage of the running, non-cryptographic decentralized social protocols and builds on it. ActivityPub [2] (Mastodon) and ATProto [3] are live systems with large user bases; both are deliberately blockchain-free—no gas fees, no wallet-gated feeds—the same practical stance web10 takes. Solid [1] goes furthest on ownership (personal data stores, WebACL) and has produced a body of work on building social applications over them. All four establish that a user can *own* their data without a ledger.

The gap web10 addresses is authorization granularity and generality. ActivityPub and ATProto authorize at the level of follows and collection-level visibility; neither has fine-grained group/role contracts or a request → approve → assign-role lifecycle, and each data model is shaped to one app. Solid's WebACL answers "who may read or write this container?"—a coarse, resource-level question—with no first-class *group* primitive carrying *roles*. web10's contribution is exactly this missing layer: access as *person* → *role* → *per-service operations*, granted *inside* a group through an explicit join-request and approval flow, where a role is a map from data service to the operations it grants (§2). On top of that, web10's node is generic—its primitives support arbitrary app shapes, not just the one social app each earlier protocol was built around—and it does so on a running platform whose feed collapses 267 requests into one.

Cloud IAM. AWS IAM [4] and the RBAC/ABAC literature [5] define the permission language web10 reuses at the user boundary: principals, roles, service-scoped operations, revocable grants. The move in this paper is directional—the same policy shape, held by the end user about their own data, enforced by the data layer rather than a control plane.

Multi-tenant isolation. The database literature on tenant isolation—schema-per-tenant, database-per-tenant, and row-level security [6]—is the closest systems ancestor. web10's key-space per user is tenant-per-end-user taken to its limit: the tenant is the *end user*, and the isolation is expressed as the query's `author_key` filter plus group-grant list rather than a physical boundary. The cost tradeoff of that choice is measured in the extended version.

6 CONCLUSION

web10 brings the cloud IAM permission model—principal classes, roles, service-scoped operations, revocable grants—to the user boundary of a social data platform, and enforces it at the query layer of a single columnar store so that fine-grained authorization costs

an index lookup, not a scan. We demonstrate the model in a running social platform whose feed is a tunable query that collapses 267 network requests into one, every row access- checked—the commercial-grade execution of user-owned social data that protocol-first systems proposed but did not ship. The extended version adds scale and latency figures and the cost comparison against row-level-security baselines.

REFERENCES

[1] T. Berners-Lee. Solid: A distributed social web protocol. *Solid Project / W3C*, 2014–present. https://solidproject.org.

[2] M. Sporny, D. Wilson, R. Smith, and D. Sadok. ActivityPub: A protocol for distributed federated publishing, archiving, and sharing. *IETF Internet-Draft*, 2023.

[3] D. Holman and M. Roth. AT Protocol: A personal data protocol for social applications. *Bluesky Technologies*, 2022–present. https://github.com/bluesky-social/atproto.

[4] Amazon Web Services. AWS Identity and Access Management (IAM) user guide. *Amazon Web Services*, 2025. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html.

[5] R. Sandhu, E. Coyne, H. Feinstein, and C. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[6] A. Kemper and S. Lott. Row-level security in PostgreSQL. *PostgreSQL Documentation*, 2024. https://www.postgresql.org/docs/current/ddl-rowsecurity.html.